Lab Assignment 4: Grammar-Aware LL(1) Syntax Analyzer

Course: CS3104 - Compiler Design Laboratory
Date: 27/08/2026

Instructions

- Write your name and roll number on top of every source file as comments.
- Source files must be named exactly: ll1.c Each program must compile cleanly with plain gcc, e.g.: gcc ll1.c -o ll1 (no parser-generator tools such as yacc/bison are to be used — the point of this lab is to build the parsing logic by hand).
- Submit only the source file through the same submission portal used for previous assignments – http://10.22.11.15/~abyaym/2026/CS3104_26/submission/

Problem Statement

A parser sits above the lexical analyzer and decides whether a stream of tokens conforms to a language's context-free grammar. A top-down (LL) parser predicts which production to expand using left context. In this assignment you will take one small expression grammar and along the way, observe exactly where ambiguity turns into a parsing conflict and how the strategy is forced to resolve it.

You will work with two versions of the same language of parenthesized **id / + /** * expressions:

```
Ambiguous grammar G1:
    E -> E + E | E * E | ( E ) | id

Unambiguous, precedence-encoded grammar G2:
    E -> E + T | T
    T -> T * F | F
    F -> ( E ) | id
```

Example

Input string (tokens already classified by your Lab 3 scanner):

```
id + id * id
```

Expected output :

```
ll1: parsing with G2 using LL(1) table...
  E -> T E'
  T -> F T'
  F -> id
  ...
  ACCEPT
```

Core Requirements

- **CFG representation:** represent each grammar as a simple in-memory (hard coded) table of productions (non-terminal, list of right-hand sides) that the program can read; do not hard-code parsing decisions outside this table where avoidable.

- **LL(1) parsing (ll1.c):** compute FIRST and FOLLOW sets for G2, build the LL(1) parsing table, and implement a table-driven predictive parser that prints the sequence of productions applied and a final ACCEPT/REJECT verdict.
- **LL(1) conflict detection:** if any parsing-table cell would need to hold more than one production, do not silently overwrite it — print "LL(1) conflict at M[<non-terminal>, <terminal>]" listing every competing production, and report that the grammar is not LL(1). (For example: G1)
- Programs must end their output with a single line reading either ACCEPT or REJECT.

Implementation Guidelines

- For FIRST/FOLLOW computation, compute FIRST for every non-terminal before FOLLOW, since FOLLOW rules depend on completed FIRST sets; iterate to a fixed point rather than assuming one pass suffices.
- When filling the LL(1) table, process every production of a non-terminal before moving to the next non-terminal, so that a genuine conflict (two productions wanting the same table cell) is detected rather than the second production quietly overwriting the first.
- Declare +, * as left-associative and give * strictly higher precedence than + in your table, matching standard arithmetic conventions.
- Reuse your Lab 3 tokenizer (or a stripped-down version of it) to turn each input line into the token stream that all three parsers consume, so lexical analysis and syntax analysis stay cleanly separated, exactly as they are in a real compiler.

Compilation & Testing

```
$ gcc ll1.c -o ll1
$ ./ll1 < input.txt
```

Deliverables

- ll1.c